EXPERIMENT-75(HDOC)

C Programming — Enumerations

S

Program 1: Enumeration for User Roles with Role-Based Messages

Step 1: Understand the Problem Statement

- Input: The user role — ADMIN, USER or GUEST (the letter case does not matter).
- Output: The message for that role: `Welcome Admin` for ADMIN, `Welcome User` for USER and `Welcome Guest` for GUEST. Any other text is rejected with an error message.
- The enumeration `enum Role { ADMIN, USER, GUEST }` gives ADMIN = 0, USER = 1 and GUEST = 2.
- `parseRole()` converts the typed text to the matching enumeration value: the text is changed to upper case with `toupper()` and compared with the three names using `strcmp()`. It returns 1 on success and 0 for an invalid role.
- `displayMessage()` receives the enumeration value and uses a `switch` statement, printing the message for `case ADMIN`, `case USER` and `case GUEST`.

Rule(s) used: ADMIN (0) → Welcome Admin, USER (1) → Welcome User, GUEST (2) → Welcome Guest. The text is converted to upper case before comparing, so "guest", "Guest" and "GUEST" all give the value 2.

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	`Input: GUEST`	Welcome Guest	Welcome Guest	Pass
2	`Input: ADMIN`	Welcome Admin	Welcome Admin	Pass
3	`Input: USER`	Welcome User	Welcome User	Pass
4	`Input: admin`	Welcome Admin	Welcome Admin	Pass
5	`Input: MANAGER`	Invalid role. Enter ADMIN, USER or GUEST.	Invalid role. Enter ADMIN, USER or GUEST.	Pass

Step 3: Algorithm

1. Start the program.
2. Declare the enumeration `enum Role` with ADMIN, USER and GUEST (values 0, 1 and 2).
3. Read the role text into `input`.
4. Call `parseRole(input, &role)`: copy the text in upper case and compare it with "ADMIN", "USER" and "GUEST" using `strcmp()`; store the matching constant in `role` and return 1, or return 0 if nothing matches.

5. If `parseRole()` returns 0 (or nothing was read), display `Invalid role. Enter ADMIN, USER or GUEST.` and stop.

6. Call `displayMessage(role)`.

7. In `displayMessage()`: print `Welcome Admin` for ADMIN, `Welcome User` for USER and `Welcome Guest` for GUEST.

8. Stop the program.

Evaluation against Step 2 test cases: the input GUEST is converted to the enumeration value 2, so the switch prints Welcome Guest. ADMIN (value 0) prints Welcome Admin and USER (value 1) prints Welcome User. The lower-case input admin becomes ADMIN after conversion and gives the same message as ADMIN, while MANAGER matches none of the three names and is rejected with the error message — identical to the Expected Output(s) column, confirming the algorithm is correct before coding.

Step 4: C Program

File: hdoc76_1.c

```c
/* HDOC Day-76, Program 1: Enumeration for user roles.

   enum Role { ADMIN, USER, GUEST } gives ADMIN = 0, USER = 1 and GUEST = 2.
   The role typed by the user is converted to the matching enumeration value
   and a switch statement displays the message for that role:
       ADMIN -> Welcome Admin     USER -> Welcome User     GUEST -> Welcome Guest
*/

#include <stdio.h>
#include <string.h>
#include <ctype.h>

/* Enumeration for the user roles */
enum Role { ADMIN, USER, GUEST };

/* Function prototypes */
int  parseRole(const char *text, enum Role *role);
void displayMessage(enum Role role);

int main(void) {
    char input[20];
    enum Role role;

    printf("Enter role (ADMIN, USER, GUEST): ");
    if (scanf("%19s", input) != 1 || !parseRole(input, &role)) {
        printf("Invalid role. Enter ADMIN, USER or GUEST.\n");
        return 1;
    }

    displayMessage(role);
    return 0;
}

/* Converts the typed role (any letter case) to the enumeration value.
   Returns 1 on success and 0 if the text is not a valid role. */
int parseRole(const char *text, enum Role *role) {
    char upper[20];
    int i;

    for (i = 0; text[i] != '\0' && i < 19; i++)
        upper[i] = (char)toupper((unsigned char)text[i]);
    upper[i] = '\0';
```

```
    if (strcmp(upper, "ADMIN") == 0)
        *role = ADMIN;
    else if (strcmp(upper, "USER") == 0)
        *role = USER;
    else if (strcmp(upper, "GUEST") == 0)
        *role = GUEST;
    else
        return 0;
    return 1;
}

/* Displays the message that belongs to the value of the enumeration */
void displayMessage(enum Role role) {
    switch (role) {
        case ADMIN:
            printf("Welcome Admin\n");
            break;
        case USER:
            printf("Welcome User\n");
            break;
        case GUEST:
            printf("Welcome Guest\n");
            break;
    }
}
```

Step 5: Compile, Execute and Evaluate

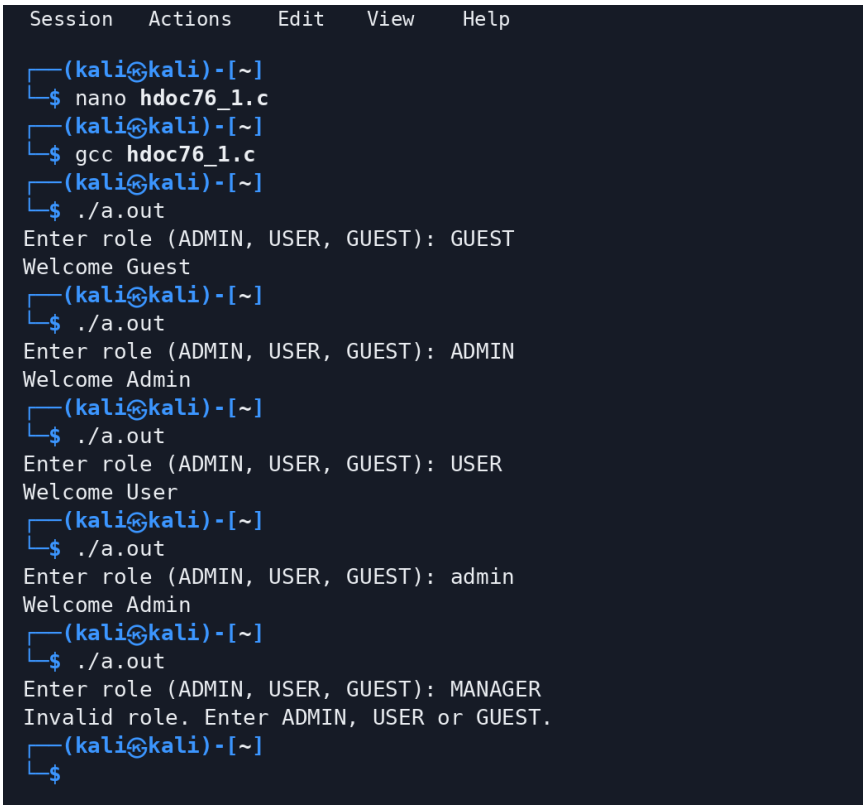
The program was compiled using the GCC compiler and executed from the Linux terminal using the following commands:

`cc hdoc76_1.c`	(compile)
`./a.out`	(execute)

All 5 test cases prepared in Step 2 were executed against the compiled program. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 6: Compilation and Execution Screenshot

Screenshot 1 of 1



Program 2: Names and Integer Values of 12 Colours Using a Loop

Step 1: Understand the Problem Statement

- Input: None — the program does not read anything.
- Output: The name and integer value of each of the 12 colours, starting with `RED=0, BLUE=1, GREEN=2` as in the question and continuing up to `CYAN=11`.
- The enumeration `enum Color { RED, BLUE, GREEN, YELLOW, ORANGE, PURPLE, PINK, BROWN, BLACK, WHITE, GRAY, CYAN }` lists 12 colours. The first name gets the value 0 and every following name gets the previous value plus 1, so the values are 0 to 11 without writing any of them.
- An enumeration does not keep its names as text, so the array of strings `colorNames[]`, written in the same order, is used to print the names; the enum value is used as the index into this array.
- A loop does the printing: the enumeration variable `color` runs from `RED` (0) to `CYAN` (11), and in every pass the program prints `colorNames[color]`, an equals sign and the integer value of `color`.

Rule(s) used: With the first enumerator RED = 0, each next colour = previous + 1, so RED = 0, BLUE = 1, GREEN = 2, ..., CYAN = 11 (12 colours). The loop `for (color = RED; color <= CYAN; color++)` therefore runs exactly 12 times and the value of a colour is also its index in `colorNames[]`.

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	No input	RED=0, BLUE=1, GREEN=2, YELLOW=3, ORANGE=4, PURPLE=5, PINK=6, BROWN=7, BLACK=8, WHITE=9, GRAY=10, CYAN=11	RED=0, BLUE=1, GREEN=2, YELLOW=3, ORANGE=4, PURPLE=5, PINK=6, BROWN=7, BLACK=8, WHITE=9, GRAY=10, CYAN=11	Pass

Step 3: Algorithm

1. Start the program.
2. Declare `enum Color` with the 12 colours RED, BLUE, GREEN, YELLOW, ORANGE, PURPLE, PINK, BROWN, BLACK, WHITE, GRAY and CYAN (values 0 to 11).
3. Call `printColors()`.
4. In `printColors()`, store the 12 colour names in the array `colorNames[]` in the order of the enumeration.
5. Set `color = RED` and repeat while `color <= CYAN`.
6. For every colour except the first, print a comma and a space; then print `colorNames[color]`, an equals sign and the value of `color`.
7. Increase `color` by 1 after each pass.
8. After the loop print a newline and stop the program.

Evaluation against Step 2 test case: the loop starts at RED (0) and prints RED=0, then BLUE=1 and GREEN=2, which are the three values shown in the question, and continues with YELLOW=3, ORANGE=4, PURPLE=5, PINK=6, BROWN=7,

BLACK=8, WHITE=9, GRAY=10 and CYAN=11. After CYAN the value 12 is greater than CYAN, so the loop ends after exactly 12 colours — identical to the Expected Output(s) column, confirming the algorithm is correct before coding.

Step 4: C Program

File: hdoc76_2.c

```c
/* HDOC Day-76, Program 2: All names and integer values of 12 colours.

   enum Color lists 12 colours. The first name gets 0 and every next name
   gets the previous value + 1, so RED = 0, BLUE = 1, GREEN = 2, ... and
   CYAN = 11. The enumeration itself does not store the names as text, so an
   array of strings in the same order is used to print them. A loop runs the
   enumeration variable from the first colour to the last one.
*/

#include <stdio.h>

/* Enumeration with 12 colours (values 0 to 11) */
enum Color { RED, BLUE, GREEN, YELLOW, ORANGE, PURPLE,
             PINK, BROWN, BLACK, WHITE, GRAY, CYAN };

/* Function prototype */
void printColors(void);

int main(void) {
    printColors();
    return 0;
}

/* Prints every colour name with its integer value using a loop */
void printColors(void) {
    /* names in the same order as the enumeration values 0 to 11 */
    const char *colorNames[] = { "RED", "BLUE", "GREEN", "YELLOW", "ORANGE",
                                 "PURPLE", "PINK", "BROWN", "BLACK", "WHITE",
                                 "GRAY", "CYAN" };
    enum Color color;

    for (color = RED; color <= CYAN; color++) {
        if (color != RED)
            printf(", ");
        printf("%s=%d", colorNames[color], color);   /* an enum value is an int */
    }
    printf("\n");
}
```

Step 5: Compile, Execute and Evaluate

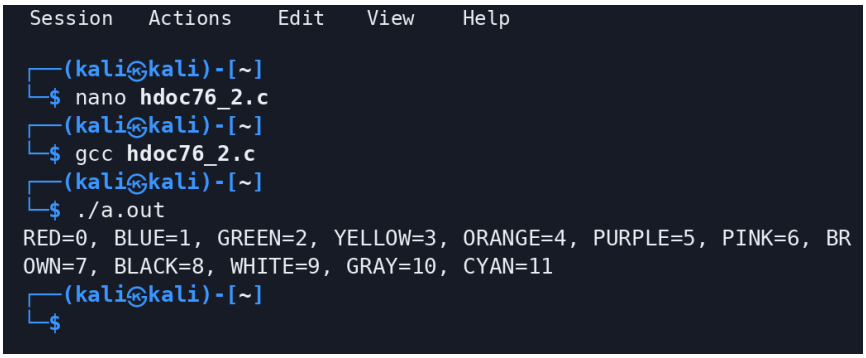
The program was compiled using the GCC compiler and executed from the Linux terminal using the following commands:

cc hdoc76_2.c	(compile)
./a.out	(execute)

All 1 test cases prepared in Step 2 were executed against the compiled program. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 6: Compilation and Execution Screenshot

Screenshot 1 of 1



Conclusion

In this experiment, enumerations were used to give meaningful names to integer constants. In the first program an enum for the user roles ADMIN, USER and GUEST (0, 1 and 2) was declared, the typed role was converted to its enum value, and a switch statement displayed the matching message (Welcome Guest for GUEST). In the second program an enum with 12 colours was declared, whose values run automatically from 0 (RED) to 11 (CYAN), and a loop with an array of names printed every colour with its integer value. Both programs were compiled with gcc and tested, including a lower-case and an invalid role, and every output matched the expected result.